IDS568 Milestone 5: LLM Inference Optimization with Batching and Caching

1. Introduction

This project focuses on improving large language model inference performance using two optimization techniques: dynamic batching and response caching. The goal was to build a production-style inference API that could reduce latency, improve throughput under concurrent load, and handle repeated requests more efficiently.

The system was implemented using FastAPI, Hugging Face Transformers, and asynchronous request handling in Python. A custom batching layer and an in-memory caching layer were added to simulate production inference optimizations in a lightweight local environment.

2. System Architecture

The final system contains the following components:

- FastAPI inference server
- Dynamic batching module
- In-memory cache with TTL and max-entry limits
- Benchmarking scripts for latency and load testing
- Visualization pipeline for result analysis

The request flow is:

Client → FastAPI API → Cache Lookup → Batcher → Model Inference → Cache Store → Response

If a repeated request is already in cache, the system returns the cached output immediately without rerunning the model. If the request is not in cache, it is submitted to the batcher and processed through the model.

3. Experimental Setup

The system was tested using the `facebook/opt-125m` model from Hugging Face. The API was implemented with FastAPI and executed locally on a MacBook Air CPU environment. Since this system was tested without a production GPU, the focus of the experiments was on correctness, concurrency behavior, and relative performance gains from batching and caching.

The main configuration settings used were:

- Model: `facebook/opt-125m`
- Maximum batch size: 8
- Batch timeout: 50 ms
- Cache TTL: 300 seconds
- Cache max entries: 1000

The benchmark suite included four main experiments:

1. Cold-cache vs. warm-cache comparison
2. Concurrent batching latency measurement
3. Cache hit-rate tracking over time
4. Multi-level synthetic load testing

The load testing was performed using low, medium, and high concurrency levels to observe how the system behaved as the number of requests increased.

4. Compute Pathways and Optimization Logic

LLM inference is expensive because generating responses requires repeated model execution over prompt input and generated tokens. In a normal non-cached setup, every request goes through the full inference path, even if an identical prompt has already been processed before.

Caching changes this pathway by allowing repeated prompts to bypass model execution completely. Instead of rerunning the model, the server returns the stored response from cache. This dramatically reduces latency for repeated requests and avoids redundant compute.

Batching changes the pathway differently. Instead of processing incoming cache misses one at a time, the batcher groups concurrent requests together before they are sent to the model. This reduces repeated per-request execution overhead and improves throughput under load. Batching is especially useful when there are several simultaneous requests that are not yet cached.

In short:

- Cache hits skip inference entirely
- Cache misses go through the batcher
- Batching improves efficiency under concurrent demand
- Caching provides the largest gains for repeated prompts

5. Benchmark Results

5.1 Cold vs. Warm Cache

The cold-cache benchmark showed a latency of about 646 ms for the first request. The warm-cache benchmark dropped to about 0.9 ms for the repeated request. This demonstrates that caching had the strongest optimization impact in the system.

This result shows that repeated prompts can be served almost instantly once cached. It also confirms that the cache logic is functioning correctly and that repeated computation is being avoided.

5.2 Batching Performance

The batching benchmark evaluated 10 concurrent requests. The average latency was about 2800 ms, with a total wall-clock time of about 3404 ms. This result shows that the server was able to process multiple requests concurrently through the batching pipeline.

Although batching did not reduce individual latency to the same degree as caching, it improved concurrent handling and better reflects how a production system would manage multiple incoming requests at once. Batching is most valuable in scenarios with concurrent cache misses.

5.3 Cache Hit Rate Over Time

The cache hit-rate benchmark showed a final hit rate of 72.5%. This confirms that repeated prompts were successfully captured and reused by the caching layer. As expected, hit rate increased over time as more repeated prompts were encountered.

This is important because the effectiveness of caching depends on workload patterns. A workload with many repeated prompts benefits significantly, while a workload of mostly unique prompts benefits much less.

5.4 Load Testing

Under low load (5 concurrent requests), the system achieved 20/20 successful responses with an average latency of about 755.7 ms.

Under medium load (20 concurrent requests), the system achieved 60/60 successful responses with an average latency of about 2.8 ms.

Under high load (50 concurrent requests), the system achieved 100/100 successful responses with an average latency of about 6.7 ms.

These results show that the API remained stable under concurrent load. The lower average latencies in medium and high load were heavily influenced by cache reuse, which demonstrates how caching can dominate performance in repeated-prompt workloads.

6. Resource Usage and System Monitoring

Because this project was tested on a local MacBook Air CPU setup, GPU utilization metrics were not available. However, the server exposes CPU and memory statistics through the `/stats` endpoint, which can be used to monitor runtime resource usage.

This is important because the milestone requires attention to resource consumption. In a GPU-backed deployment, the same monitoring design could be extended to track GPU utilization and memory pressure more directly.

In this local setup, CPU and memory tracking still provide useful operational visibility and support the idea of a production-style inference service.

7. Trade-off Analysis

The results show that batching and caching solve different performance problems.

Caching provides the largest latency improvement, but only for repeated prompts. If prompts are unique, the cache contributes little benefit.

Batching improves throughput and concurrent handling, but it introduces a trade-off. A larger batch size may improve efficiency, but waiting for more requests can increase latency for individual users. Similarly, a shorter timeout reduces waiting time but may produce smaller, less efficient batches.

Cache design also introduces trade-offs. A larger cache can improve hit rate, but it uses more memory. A longer TTL can improve reuse, but it also increases the risk of stale responses remaining available for too long.

These trade-offs show why production systems must tune batching and caching based on workload rather than applying fixed settings blindly.

8. Scaling Strategy

Several improvements would make this system more production-ready.

First, the in-memory cache could be replaced with Redis to support distributed deployment and stronger persistence controls. Second, inference could be moved to GPU-backed infrastructure to better evaluate throughput and utilization under realistic LLM workloads. Third, continuous batching frameworks such as vLLM or Text Generation Inference could improve request scheduling efficiency beyond this lightweight custom implementation.

Additional improvements could include adaptive batching, semantic caching, and autoscaling based on queue depth or request volume. These changes would allow the system to handle larger workloads more efficiently while keeping latency under control.

9. Conclusion

This project demonstrates that dynamic batching and caching can significantly improve inference system performance, but in different ways. Caching provides the most dramatic latency reduction for repeated prompts, while batching improves the handling of concurrent requests and supports better throughput under load.

The final system successfully implemented a modular FastAPI inference server with configurable batching and caching, reproducible benchmark scripts, result visualizations, and supporting analysis. Overall, the project shows how production-style LLM inference optimization requires balancing latency, throughput, resource efficiency, and governance concerns.